第4章 串

- 4.1 串及串的基本操作
- 4.2 串存储和实现
- 4.3 串的模式匹配算法
- 4.4 串操作应用举例

§4.1 串类型的定义

- 计算机上的非数值处理的对象基本上是字符串数据。
- 串的概念

串是由零个或多个字符组成的有限序列，一般记作

S = ‘a1, a2, a3, . . . an’

串的例子：

- （1）a = ‘LIMING’
- （2）b = ‘PEJING UNIUERCITY’
- （3）c = ‘DATA STRUCTURE’
- （4）d = ‘’
- （5）e = ‘ ’

串的有关术语

- 子串

  c = ‘ DATA STRUCTURE’ ,
  f= ‘DATA’

- 子串的位置

  子串T在主串S中的位置是指主串S中第一个与T相同的子串的首字母在主串中的位置;

  例：S= ‘ababcabcac’ , T= ‘abc’

串的有关术语

- 串相等

  只有当两个串的长度相等，并且各个对应位置的字符都相等时两个串才相等。

- 注意：

  串值必须用一对单引号括起来，但单引号本身不属于串。

串的抽象数据类型定义

ADT String {

数据对象：

$D=\{a_i \mid a_i \in CharacterSet,\ i=1,2,\ldots,n,\ n\geq 0\}$

数据关系：

$R1=\{ <a_{i-1}, a_i> \mid a_{i-1}, a_i \in D,\ i=2,\ldots,n \}$

基本操作：……

} ADT String

串的基本操作

- 串赋值操作

StrAssign(&T, chars)

初始条件：chars是字符串常量

操作结果：生成一个其值等于chars的串T

- 复制串操作

StrCopy(&T,S)

初始条件：串S存在

操作结果：由串S复制得到串T；

串的基本操作

- 判空操作

StrEmpty(S)

初始条件：串S存在

操作结果：

若S为空串，则返回TRUE，否则返回FALSE

串的基本操作

- 串比较操作

  StrCompare(S, T)

  初始条件：串S和T存在

  操作结果：若S>T，则返回值>0；

  若S=T，则返回值=0；

  若S<T，则返回值<0；

串的基本操作

- 串置空操作

  ClearString(&S)

  初始条件：串S存在

  操作结果：将S清为空串

- 求串长操作

  StrLenght(&S)

  初始条件：串S存在

  操作结果：返回S的元素个数(串的长度)

串的基本操作

- 串连接操作

Concat(&T, S1, S2)

初始条件：串S1和S2存在

操作结果：

用T返回由S1和S2连接成的新串

串的基本操作

- 求子串操作

SubString(&Sub, S, pos, len)

初始条件：串S存在，

1≤pos≤StrLength(S)

且 0≤len≤StrLength(S)-pos+1

操作结果：用Sub返回串S第pos个字

串的基本操作

- 求子串位置操作

Index(S, T, pos)

初始条件：串S和T存在，T是非空串

$$1 \leqslant pos \leqslant StrLength(S)$$

操作结果：返回S中第pos个字符之后与T相同的子串的位置，若不存在返回0

串的基本操作

- 串替换操作

Replace(&S, T, V)

初始条件：串S, T和V存在，T是非空串

操作结果：用V替换主串S中出现的所有与T相等的不重叠的子串

串的基本操作

- 串插入操作

StrInsert(&S, pos , T)

初始条件：串S和T存在，

$$1 \leqslant pos \leqslant StrLength(S)+1$$

操作结果：在串S的第pos个字符之前插入串T

串的基本操作

- 串删除操作

StrDelete(&S, pos , len)

初始条件：串S存在

$1 \leqslant pos \leqslant StrLength(S)-len+1$

操作结果：从串S中删除第pos个字符起长度为len的子串。

串的基本操作

- 串销毁操作

  DestroyString(&S)

  初始条件：串S存在

  操作结果：串S被销毁。

§4.2 串的表示和实现

- 串有三种机内表示方法：
- 1 定长顺序存储表示

类似于线性表的顺序存储结构，用一组地址连续的存储单元存储串值的字符序列。在串的定长顺序存储结构中，按照预定义的大小，为每个定义的串变量分配一个固定长度的存储区。

定长顺序存储表示

- 用定长数组如下描述：

```
#define  MAXSTRLEN  255
// 用户可在255以内定义最大串长
typedef  unsigned  char
                SString[MAXSTRLEN +1]
// 0号单元存放串的长度
```

S

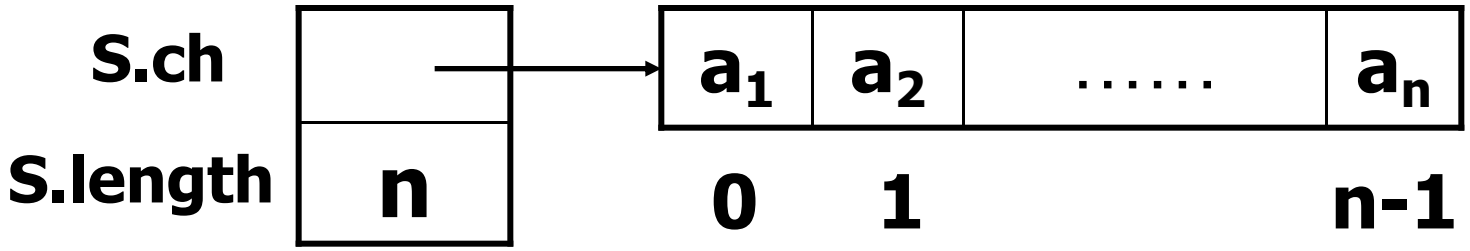
n	a_1	a_2	a_3	……	a_n		
0	1	2	3		n		MAXSTRLEN

串的堆分配存储

以一组地址连续的存储单元存放串值字符序列，但它们的存储空间是在程序执行过程中动态分配而得。

串的堆分配存储

```
typedef  struct {
 char *ch ; //存放串值的存储空间基址
 int  length ; // 存放串长
} HString ;
```



Ⅰ 串插入操作

```
Status StrInsert(HString &S,int pos,HString T)
{ //1≤pos≤StrLength(S)+1,在串S第pos个字符前插入串T
  if(pos<1||pos>S.length+1)  return ERROR ;
  if(T.length) {
    if(!(S.ch=(char *)realloc(S.ch,(S.length+
       T.length)*sizeof(char))))     exit(OVERFLOW);
    for(i=S.length-1;i>=pos-1;--i)
        S.ch[i+T.length]=S.ch[i];
    S.ch[pos-1..pos+T.length-2]=T.ch[0..T.length-1];
    S.length+=T.length;    }
 return OK;     }
```

II 串生成操作

```
Status StrAssign(HString &T,char * chars)
{ //生成一个其值等于串常量chars的串T
   if(T.ch)  free(T.ch) ;
   for( i=0 , c=chars ; c ; ++i , ++c ) ;
   if( !i ) { T.ch=NULL ; T.length=0 ; }
   else {   if( !( T.ch=(char*)malloc(i*sizeof(char))))
                  exit(OVERFLOW);
            T.ch[0..i-1] = chars[0..i-1] ;
            T.length=i ;    }
return OK;     }
```

III 求串长度操作

```
int  StrLength ( HString S )
{  //返回S的元素个数
   return  S.length ;
}
```

Ⅳ 串比较操作

```
int StrCompare(HString S , HString T )
{  //若S>T则返回值>0;若S=T则返回值=0;
   //若S<T则返回值<0
   for(i=0;i<S.length&&i<T.length;++i)
     if(S.ch[i]!=T.ch[i])  return S.ch[i]-T.ch[i];
   return  S.length-T.length ;
}
```

V 清除串操作

```
Status ClearString( HString &S )
{  // 将S清为空串
    if( S.ch )
      { free( S.ch ) ; S.ch=NULL ; }
   S.length=0 ;
    return  OK ;   }
```

Ⅵ 串连接操作

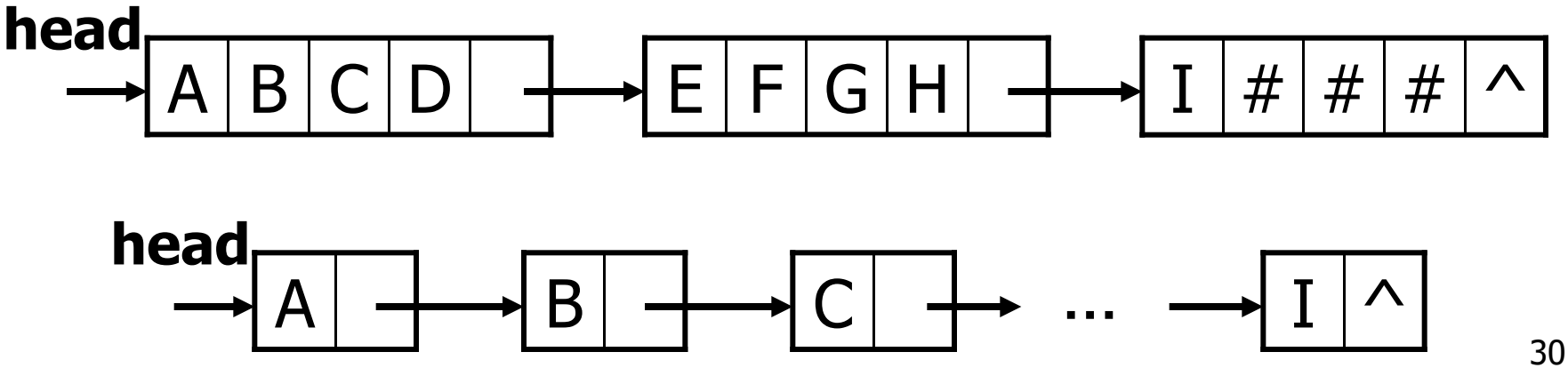
```
Status Concat(HString &T,HStirng s1,HString s2)
{ //用T返回由S1和S2连接而生成的新串
  if( T.ch )  free( T.ch ) ;
  if( !
  ( T.ch=(char*)malloc((S1.length+S2.length)
        * sizeof (char))))      exit( OVERFLOW ) ;
  T.ch[0..S1.length-1] = S1.ch[0..S1.length-1] ;
  T.length=S1.length+S2.length ;
  T.ch[S1.length..T.length-1]=
                    S2.ch[0..S2.length-pos-1] ;
  return OK;     }
```

Ⅶ 求子串操作

```
Status SubString(HString &Sub , HString S,
        int pos ,  int len)
{ //1≤pos≤StrLength(S)且0≤len≤StrLength(S)-pos+1
  if( pos<1 || pos>S.length || len<0 ||
     len>S.length-pos+1 )      return  ERROR ;
  if(Sub.ch)  free( Sub.ch ) ;
  if(!len){Sub.ch=NULL;Sub.length=0;}
  else{Sub.ch=(char*)malloc(len*sizeof(char));
        Sub.ch[0..len-1]=S.ch[pos-1..pos+len-2];
        Sub.length = len ;  }
  return OK;     }
```

串的块链存储表示

- 和线性表的链式存储结构类似，可以采用链表方式存储串值。
- 用链表存储串值时，每个结点可以存放一个字符，也可以存放多个字符。



为了便于进行串的操作，当以链表存储串值时，除头指针外还可附设一个尾指针指示链表中的最后一个结点，并给出当前串的长度。称如此定义的串存储结构为块链结构。

```
#define CHUNKSIZE 80
typedef struct Chunk {
    char ch[CHUNKSIZE] ;
    struct Chunk * next ;
} Chunk ;
typedef struct {
    Chunk * head, * tail ;
    int curlen ;
} LString ;
```

§4.3 串的模式匹配算法

1. 求子串位置的定位函数

子串的定位操作通常称为串的**模式匹配**，子串称为**模式串**。

```
int Index(SString S, SString T, int pos ) {
// 返回主串S第pos个字符后子串T的位置。若不存在，函数值为0
// T非空，1≤pos≤StLength(S)
 i=pos ;   j=1 ;    // i=pos,主串匹配的起始位置
 while (i<=S[0] && j<=T[0])  {
   if(S[i]==T[j]){ ++i; ++j;} //继续比较后继字符
   else  { i=i-j+2 ; j=1 ; }
   // 指针i后退(至当前匹配位置的下一位置)重新开始匹配
 }
 if(j>T[0])  return(i- T[0]) ; //匹配成功
 else  return 0 ;
}// Index
```

匹配过程

```
主串        a b a b c a b c a c b a b
模式        a b c a c
                  ↓i=3
第一趟匹配  a b a b c a b c a c b a b
            a b c
                ↑j=3
              ↓i=2
第二趟匹配  a b a b c a b c a c b a b
              a
              ↑j=1
                          ↓i=7
第三趟匹配  a b a b c a b c a c b a b
                a b c a c
                          ↑j=5
```

第四趟匹配

i=4

a b a b c a b c a c b a b

a

j=1

第五趟匹配

i=5

a b a b c a b c a c b a b

a

j=1

第六趟匹配

i=11

a b a b c a b c a c b a b

a b c a c

j=6

算法时间复杂度分析

主串　　‘aaaaaaaaaaaaaab’

模式串　‘aaaaaaab’

主串长度为n，模式串长度为m

此情况下while循环执行频度为

$$(n-m+1)*m$$

时间复杂度为　$O((n-m+1)*m)$

最坏情况下时间复杂度为 $O(n*m)$

2. 模式匹配改进算法——KMP匹配算法

KMP算法的改进在于：

每当一趟匹配过程中出现字符比较不等时，不需回溯 i 指针，而是利用已经得到的“部分匹配”的结果将模式向右移动尽可能远的一段距离后，继续比较。

KMP算法匹配过程示例

主串　a b a b c a b c a c b a b

模式　a b c a c

i=3

第一趟匹配　a b a b c a b c a c b a b

a b c

j=3

i=3 → i=7

第二趟匹配　a b a b c a b c a c b a b

a b c a c

j=1 → j=5

i=7 → i=11

第三趟匹配　a b a b c a b c a c b a b

a b c a c

j=2 → j=6

- 在KMP算法整个匹配过程中，i指针没有回溯操作。
- 需要解决的问题：

  当主串中第i个字符与模式中第j个字符“失配”（即比较不相等）时，主串中第i个字符应该与模式中哪个字符再进行比较。

定义函数**next[j]**，其作用为当模式中第**j**个字符与主串中相应字符“失配”时，在模式中需要重新和主串中该字符进行比较的字符的位置。

$$
next[j]=\begin{cases} 0 & \text{当} j=1 \text{时} \\ Max\{k \mid 1<k<j \text{且} `p_1 \ldots p_{k-1}'=`p_{j-k+1} \ldots p_{j-1}'\} & \\ 1 & \text{其他情况} \end{cases}
$$

next[j]函数举例

j	1	2	3	4	5	6	7	8
模式串	a	b	a	a	b	c	a	c
next[j]	0	1	1	2	2	3	1	2

注意：next函数值仅取决于模式串本身，而和相匹配的主串无关。

利用模式的next函数匹配过程示例

第一趟匹配

i=2

a c a b a a b a a b c a c a a b c

a b

j=2 next[2]=1

第二趟匹配

i=2

a c a b a a b a a b c a c a a b c

a

j=1 next[1]=0

利用模式的next函数匹配过程示例

第三趟匹配

i=3 → i=8

a c a b a a b a a b c a c a a b c

a b a a b c

j=1 → j=6 next[6]=3

第四趟匹配

i=8 → i=14

a c a b a a b a a b c a c a a b c

a b a a b c a c

j=3 → j=9

KMP算法

```
int Index_KMP ( SString S, SString T,
        int pos )   {
  i = pos ; j=1;
  while ( i <=S[0] && j <=T[0]) {
     if (j==0|| S[i]==T[j] )
        {  i++;   j++;  }
     else  j = next[ j ] ;     }
  if(j>T[0])   return i-T[0] ;
  else  return 0;  }
```

计算next函数的方法

- 【Method 1】按定义

$$
next[j]=\begin{cases} 0 & \text{当}j=1\text{时} \\ Max\{k \mid 1<k<j \text{且} `p_1 \ldots p_{k-1}'=`p_{j-k+1} \ldots p_{j-1}'\} \\ 1 & \text{其他情况} \end{cases}
$$

j	1	2	3	4	5	6	7	8	9	10
模式串	a	b	c	d	a	b	c	a	c	d
next[j]	0	1	1	1	1	2	3	4	2	1

计算next函数的方法

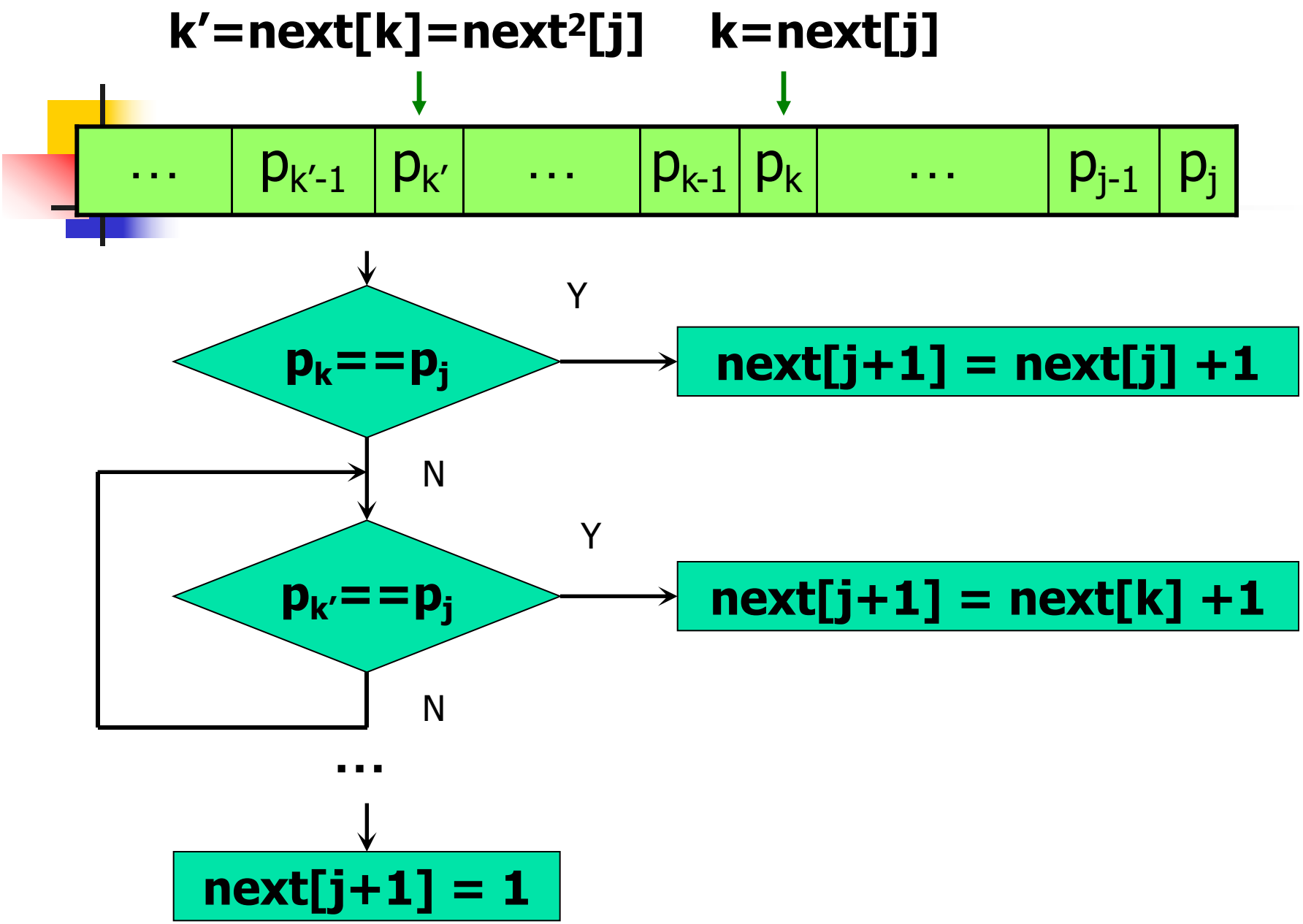
- 【Method 2】

由定义已知 next[1]=0

设next[j]=k，其中1<k<j，则有

$$‘p_1p_2\cdots p_{k-1}’=‘p_{j-k+1}\cdots p_{j-1}’$$

求next[j+1]=?


k′=next[k]=next²[j]
k=next[j]
… $p_{k'-1}$ $p_{k'}$ … p_{k-1} p_k … p_{j-1} p_j
$p_k==p_j$
Y
next[j+1] = next[j] +1
N
$p_{k'}==p_j$
Y
next[j+1] = next[k] +1
N
…
next[j+1] = 1

求next函数值算法

```
void get_next ( SString T, int next[ ])
{ // 求模式串T的next函数值并存入数组next
  i=1; next [ 1 ] =0; j=0;
  while( i < T[0]) {
    if (j==0|| T[i]==T[j] )
      { ++i ; ++j ; next[ i ]=j ; }
    else j=next[ j] ;    }
}// get_next
```

对next函数的改进

前面定义的next函数尚有缺陷：

主串 a a a b a a a a b

模式 a a a a b

j	1	2	3	4	5
模式	a	a	a	a	b
next[j]	0	1	2	3	4
nextval[j]	0	0	0	0	4

改进算法

```
void get_nextval ( SString T, int next[ ])
{ // 求模式串T的next函数修正值存入数组next
  i=1;  nextval[1] =0;  j=0;
  while( i < T[0])  {
    if (j==0|| T[i]==T[j] )
      {  ++i ;  ++j ;
         if(T[i]!=T[j]) nextval[i]=j ;
         else  nextval[i]=nextval[j]  }
    else  j=next[j] ;     }
}// get_next
```